LABORATORIO

Visualización de Datos con Grafana

preparado por: Antonio Russoniello

1. Introducción

Grafana es una herramienta de código abierto diseñada para la visualización y monitoreo de datos en tiempo real. Es ampliamente utilizada para crear paneles interactivos y dashboards que permiten analizar métricas, logs y otros tipos de datos provenientes de diversas fuentes, como bases de datos, sistemas de monitoreo, APIs y más.

Grafana es especialmente popular en el ámbito de la observabilidad, donde se utiliza para supervisar infraestructuras, aplicaciones y servicios. Su flexibilidad y capacidad para integrarse con múltiples fuentes de datos lo convierten en una herramienta esencial para el monitoreo y el análisis de datos en redes IoT.

2. Características principales de Grafana

2.1. Visualización de datos:

- Grafana permite crear gráficos, tablas, mapas de calor y otros tipos de visualizaciones personalizadas
- Soporta una amplia variedad de tipos de gráficos, como líneas, barras, gauges, histogramas, entre otros

2.2. Integración con múltiples fuentes de datos:

- Bases de datos temporales como Prometheus, InfluxDB y TimescaleDB
- Sistemas de logs como Elasticsearch, Loki y Splunk.
- APIs y servicios en la nube como AWS CloudWatch, Google Cloud Monitoring y Azure Monitor
- Bases de datos relacionales como MySQL, PostgreSQL y SQL Server

2.3. Dashboards interactivos

- Los dashboards de Grafana son altamente personalizables y permiten interactuar con los datos en tiempo real
- Puedes agregar paneles, configurar alertas y compartir dashboards con otros usuarios

2.4. Alertas y notificaciones

- Grafana incluye un sistema de alertas que permite configurar reglas basadas en métricas específicas
- Las alertas pueden enviarse a través de canales como email, Slack, PagerDuty, Telegram y otros

3. Casos de uso comunes

3.1. Monitoreo de infraestructura:

- Supervisar servidores, redes, contenedores y clusters de Kubernetes
- Visualizar métricas como uso de CPU, memoria, tráfico de red y latencia

3.2. Monitoreo de Aplicaciones:

- Monitorear el rendimiento de aplicaciones y microservicios
- Analizar logs y trazas para detectar errores y cuellos de botella

3.3. Análisis de datos en tiempo real:

- Visualizar datos generados por sensores IoT, dispositivos o sistemas de producción
- Crear dashboards para analizar tendencias y patrones

3.4. Reportes y presentaciones:

- Generar reportes visuales para presentar métricas clave a equipos o stakeholders
- Compartir dashboards con otros usuarios o exportarlos como imágenes o PDF

4. Instrucciones

En este laboratorio haremos uso del broker mosquitto instalado y configurado en el laboratorio anterior, este paso debe realizarse previo a los pasos indicados para la instalación de Grafana.

4.1. Actualizar sistema

```
sudo apt update && sudo apt upgrade -y
```

Instalar protocolo de tiempo de red NTP:

```
sudo apt-get install -y ntpsec
```

```
sudo systemctl status ntpsec --no-pager
```

4.2. Instalar docker y docker compose

```
sudo rm -f /etc/apt/sources.list.d/docker.list
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor \
-o /etc/apt/keyrings/docker.gpg
```

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/debian \
bookworm stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
sudo apt-get update
```

```
sudo apt-get install -y docker-ce \
 docker-ce-cli containerd.io docker-compose-plugin
```

```
sudo usermod -aG docker $USER && \
 sudo docker --version && sudo docker compose version
```

habilitar docker en modo rootless:

```
sudo setfacl -m user:$USER:rw /var/run/docker.sock
```

4.3. Descargar imágenes y levantar los contenedores

se instalarán los contenedores de grafana y prometheus, aunque este último no será utilizado en este laboratorio se procederá a instalarlo para uso de los próximos laboratorios.

Crear archivo `docker-compose.yml` (**nota: los paths deben ser absolutos**)

```
services:
 prometheus:
  image: prom/prometheus:latest
  ports:
   - "9090:9090"
  volumes:
   - /home/usuario/contenedor/prometheus.yml
  command: --config.file=/etc/prometheus/prometheus.yml
 grafana:
  image: grafana/grafana
  ports:
   - "3000:3000"
  volumes:
   - /home/usuario/contenedor/grafana.yml
  links:
   - "prometheus"
```

crear archivo `prometheus.yml`

```
global:
   scrape_interval: 5s
   evaluation_interval: 15s
rule_files:
scrape_configs:
   - job_name: 'prometheus'
static_configs:
   - targets: ['localhost:9090']
```

modificar los permisos de los archivos `yml` creados en los pasos anteriores:

```
chmod 755 *
```

levantar los contenedores:

```
docker compose up -d
```

4.4. Acceder a interface de Grafana y Prometheus

Habilite los puertos de Mosquitto, Grafana y Prometheus en el servidor.

Desde el browser iniciar prometheus: `http://ip_servidor:9090`

Iniciar grafana: `http://ip_servidor:3000` → `user: admin y password: admin`

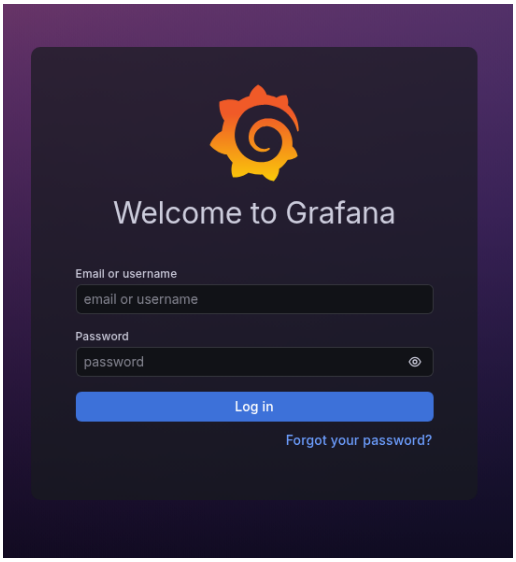


Figura 1: Interface Grafana

Configurar Grafana

Ir a *connections* y agregar una conexión a MQTT, para ejecutarlo desde el host local debe colocarse la dirección localhost como 0.0.0.0

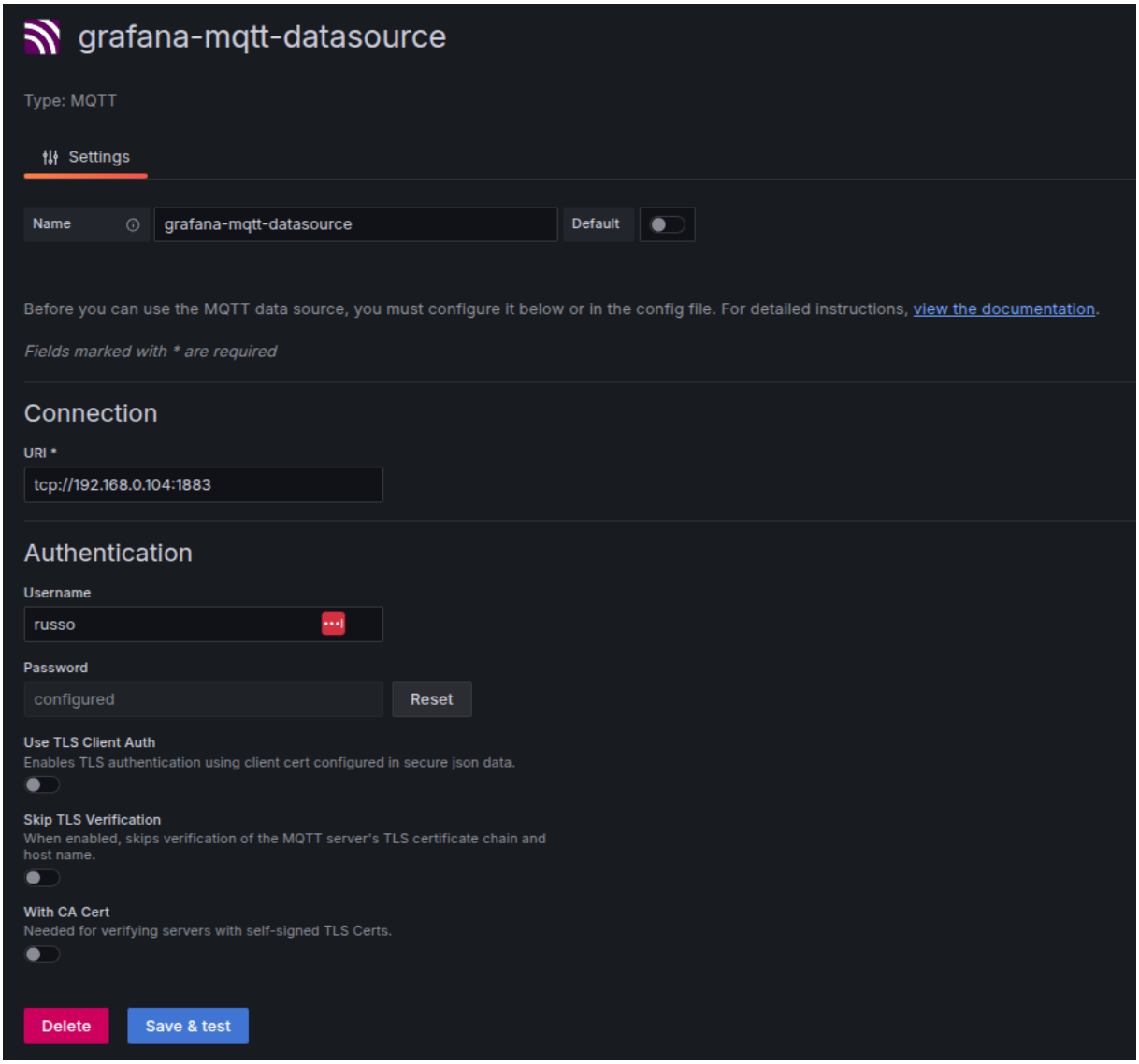


Figura 2: Conectar con el Broker MQTT

ir a dashboard para la visualización de tópicos:

5. Actividades

5.1. Conecte el sensor de Ultrasonidos a la placa. Programe un script en Python que sea capaz de recibir los datos enviados por el sensor. Muestre el circuito en Fritzing. (**5 puntos**)

5.2. El script debe publicar los datos recibidos como tópicos y enviarlos al broker mosquitto. Siga la siguiente nomenclatura para los tópicos `/NombreGrupo/distancia`, `/NombreGrupo/temperatura`, `/NombreGrupo/humedad` (puede utilizar herramientas como *MQTT Explorer* para verificar la conexión al broker y la publicación correcta de los tópicos). Visualice sus tópicos en Grafana. (**5 puntos**)

5.3. Visualice los tópicos de otros grupos en su dashboard de Grafana. (**6 puntos**)

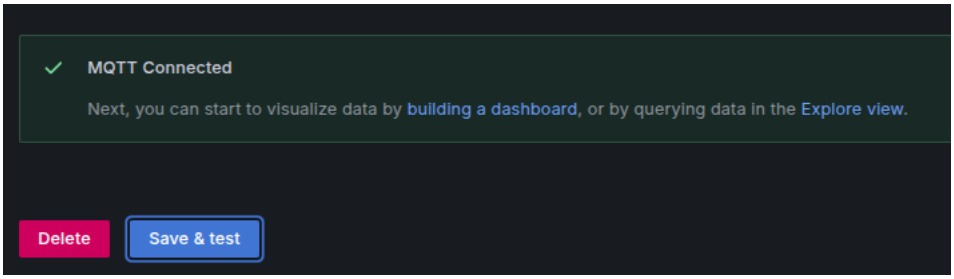


Figura 3: Probar Conexión al Broker MQTT

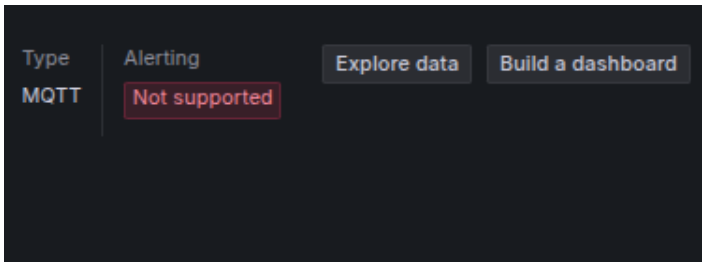


Figura 4: Dashboard

5.4. Investigue como enviar alertas a un bot Telegram utilizando los puntos de contacto de grafana, para esto deberá crear su propio bot utilizando `@BotFather`. (**4 puntos**)

§

Total de puntos: 20

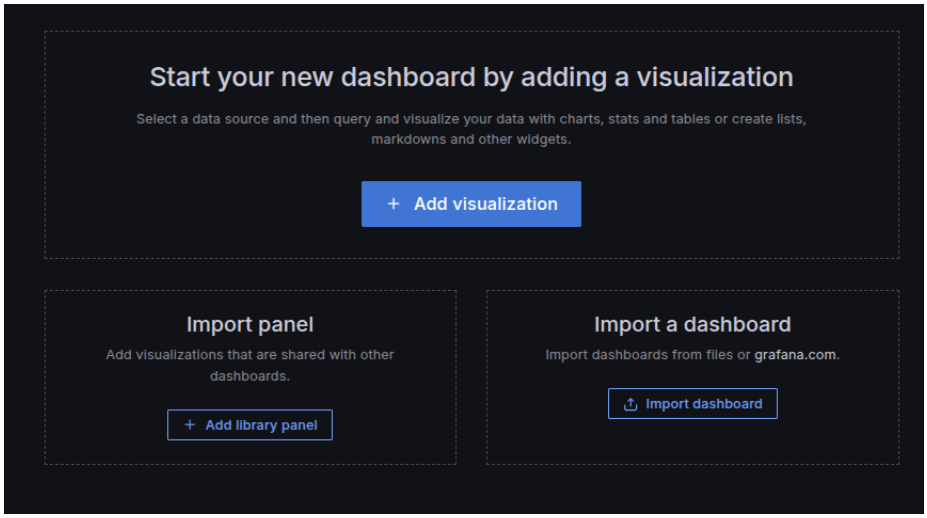


Figura 5: Agregar Visualización al Dashboard

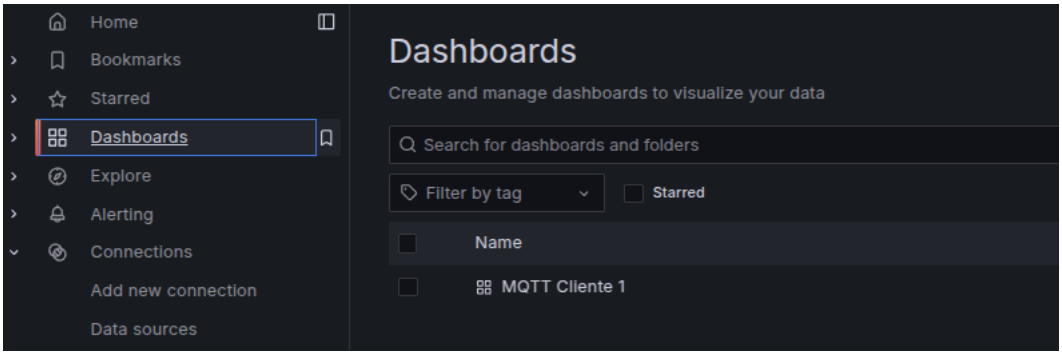


Figura 6: Seleccionar Dashboard a Visualizar

Bibliografía

- Salituro, Eric, Learn Grafna 10.x, 2nd Edition, edit. packt. 2023
- https://grafana.com/docs/grafana/latest/setup-grafana/installation/docker/
- https://mqtt-explorer.com/



Figura 7: Visualización de Tópicos